

Chapter 1

L3 - Storage, Files, and Indexing

Extra: SQL in the era of AI

Large language models (LLMs) make one question unavoidable: will natural language (NL) replace SQL? Will people soon query databases in plain English?

NL2SQL systems. AI systems that turn natural language into SQL queries. Examples include Palimpzest and Lotus. They act as a *front-end*: they help users ask, draft, and explain queries, but they do not replace the database itself.

Databases and LLMs play different roles:

- **DBMS (Back-end)**: Stores data, enforces constraints, manages transactions, controls access, and executes queries efficiently.
- **LLM (Front-end)**: Turns prompts into text or code, but gives **no correctness guarantees**.

Current limitations of NL2SQL

These tools are useful, but they still need careful checking:

- People still outperform the best current systems (for example, 93% vs. 82% on the BIRD benchmark).
- Execution accuracy drops significantly as query difficulty increases.
- Common AI errors include wrong joins, wrong aggregation level, missing **DISTINCT**, and wrong filter order.

Why relational reasoning still matters

SQL and the relational model give **precision, composability, and optimization**. Even if natural language becomes the visible interface, the database still runs on these ideas.

If you do not understand SQL and relational reasoning, you cannot:

- **Verify** the correctness of AI-generated queries.
- **Debug** query failures or logical errors.
- **Predict** and optimize database performance.

1.1 File and access layer

At the physical level, a database is a **file of records**. The DBMS stores these records in operating-system files and supports a small set of access operations.

The main operations at this layer are:

- create and delete files,
- insert, delete, and update records,
- **point access**: retrieve one record via its identifier,
- **range access**: retrieve all records satisfying a condition,
- **scan**: read the entire file.

1.1.1 File organization and indexing

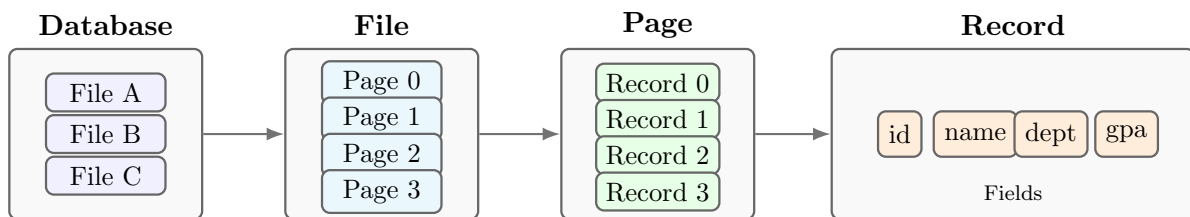
This layer answers three questions:

- how records are stored in files,
- how indexes speed up access,
- how pages are placed on storage.

Metadata about these choices is stored in the *system catalog*.

Storage hierarchy. Database storage has levels:

- a **file** is a set of **pages** (blocks),
- a **page** stores multiple **records**,
- a **record** is a sequence of **fields**.



Physical design means choosing a file organization, managing free space, and adding indexes for fast access.

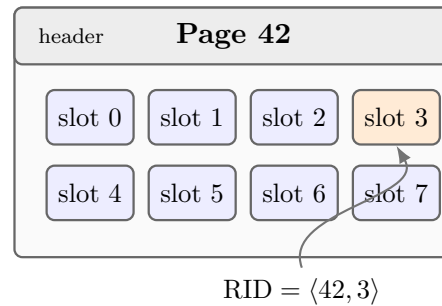
1.2 Page and record formats

Most databases use the N-ary storage model (row store): each page stores many complete tuples, one per slot.

Record Identifier (RID). A record is usually identified by its page id and slot number, often written $\langle \text{page_id}, \text{slot_number} \rangle$. Some systems show a logical identifier instead, but it still maps to a page and a slot.

A page format must support search, insertion, and deletion efficiently. The first question is whether records are **fixed-length** or **variable-length**.

Once the DBMS finds the right page, the slot number points to the record inside that page. **Keeping this identifier stable is one of the main goals of page design.**

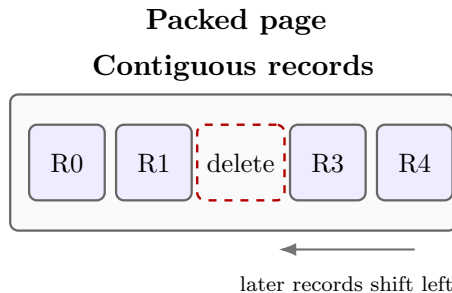


1.2.1 Fixed-length records

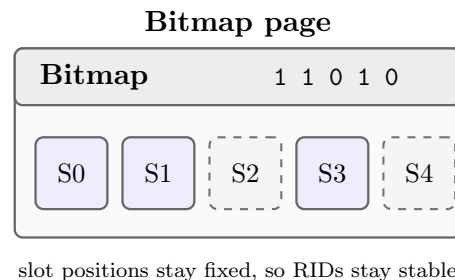
If all fields have fixed size, for example integers or fixed-size **char** values, then every record has the same length:

- the schema is fixed and stored in the catalog,
- field i starts at a fixed offset,
- pages can use very simple slot layouts.

Fixed-length page layouts



Records are stored next to each other, so space usage is very good. The downside is that deleting one record shifts later records and changes their RIDs.



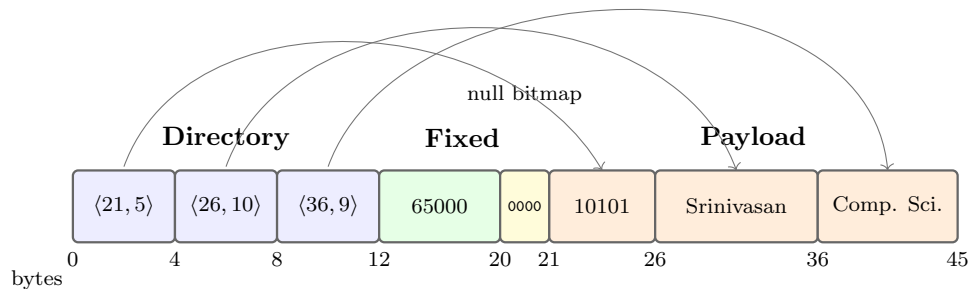
A bitmap shows which fixed slots are in use. When a record is deleted, its bit is cleared. The empty slot can be reused later, and record positions stay stable.

1.2.2 Variable-length records

We need variable-length layouts when records contain fields such as **varchar**, or when one file stores several record types.

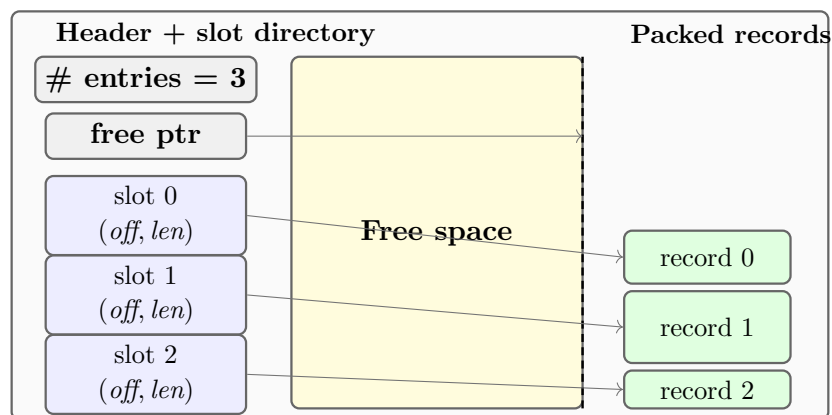
A common layout contains:

- fixed-size values at the front,
- one $\langle \text{offset}, \text{length} \rangle$ entry per variable field,
- the actual variable-length data at the end,
- a small null bitmap for missing attributes.



Slotted page structure

To store variable-length records, most systems use a **slotted page**. The slot number stays fixed even if the record itself moves within the page.



The header and slot directory grow from the front of the page. The records grow from the back. A free-space pointer marks the boundary between them. During compaction, the DBMS can move the record bytes without changing the slot number, so the RID stays the same.

The header stores:

- the number of slot entries,
- a free-space pointer,
- one slot entry per record with its location and size.

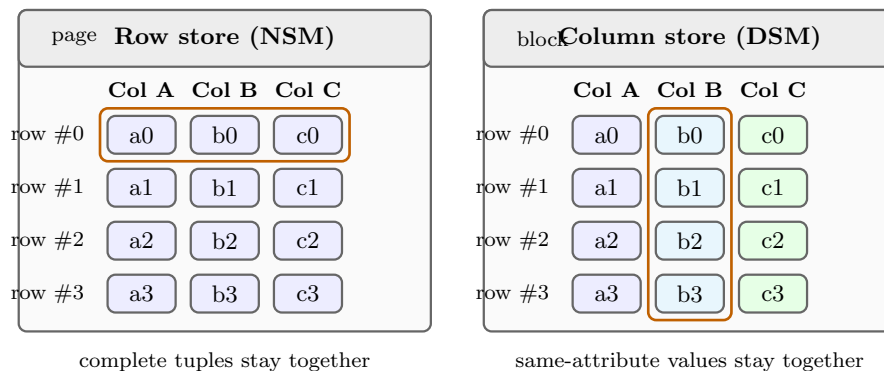
Operational issues

- **record growth**: updating a field may force data to shift within the page,
- **page overflow**: if the record no longer fits, it may move to another page, often with a forwarding pointer,
- **oversized records**: some systems forbid records larger than a page, while others spill large values to overflow storage.

1.3 Row- and column-oriented layouts

The page layouts above use the classic row-store design: each slot points to one full tuple. This is also called the **N-ary storage model (NSM)**.

Decomposition storage model (DSM). A column-oriented layout where each attribute is stored separately. To rebuild a tuple, the system reads the values at the same logical position from the needed columns.



1.3.1 Row store versus column store

Row stores keep all attributes of a tuple together. They work well when queries read or update whole records.

- **row-store advantages**: fast inserts, updates, and deletes; good when workloads read or rewrite full tuples; works well with clustered, index-based storage.
- **row-store disadvantages**: inefficient when a query scans many tuples but needs only a few attributes; compression is weaker because one page mixes many value types.

Column stores make the opposite trade-off. They optimize scans over attributes instead of full tuples.

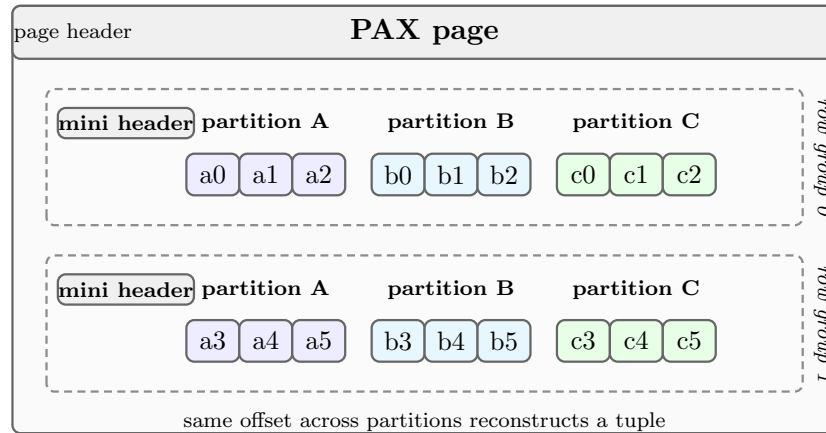
- **column-store advantages**: reduced I/O when only some attributes are needed, better CPU cache locality, stronger compression, and efficient vectorized processing.
- **column-store disadvantages**: full-tuple reconstruction is expensive, updates and deletions are harder, and compressed data may need decompression before processing.

As a rule of thumb, **row stores** fit transactional workloads (**OLTP**), while **column stores** fit analytical workloads (**OLAP**). Some systems support both and act as **hybrid row/column stores**.

1.3.2 Partition Attributes Across (PAX)

Pure column stores separate attributes across the whole file. **PAX** is a middle ground: from the outside the page still looks like a row-store page, but inside the page the values are grouped by column.

PAX. Within one page, values of the same attribute are stored together in column partitions. A tuple is rebuilt logically by taking the same offset in each partition.



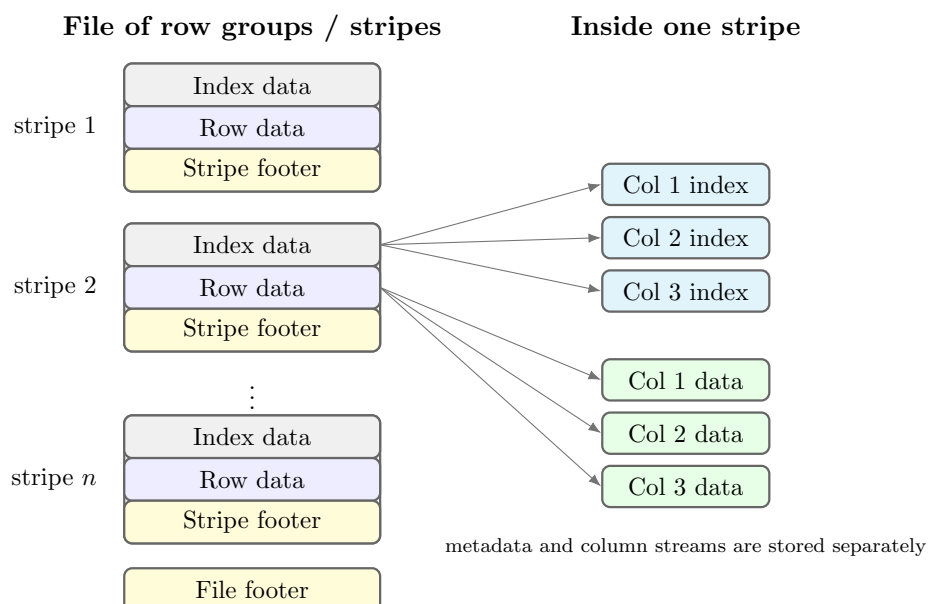
A PAX page keeps:

- the same number of tuples per page as a traditional row page,
- one contiguous partition per attribute inside the page,
- logical rather than physical row boundaries.

This improves cache behavior for scans over a few attributes without giving up the page-based design already used by row stores.

1.3.3 Columnar file formats

Modern analytical file formats apply the same idea at the file level. Formats such as **Parquet** and **ORC** group tuples into large row groups (or stripes) and store each group column by column.



These formats are popular in big-data systems because:

- each column stream compresses well,

- the engine can skip irrelevant columns and often entire groups,
- scans over a subset of attributes avoid touching the rest of the file.

1.4 Alternative file organizations

So far, we looked at how records are stored *inside* pages. Another design choice is how pages are arranged *inside* a file.

- **heap files**: good when typical access is a full file scan or when insertion simplicity matters most,
- **sorted files**: good when records are often read in order or by key range,
- **index-based organizations**: discussed in the indexing part.

1.4.1 Heap files

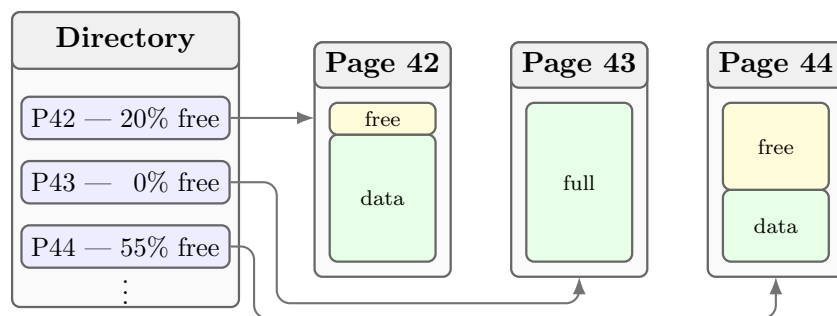
Heap file. An unordered file organization: records are stored in no particular search order.

Heap files are the simplest file organization.

- records can be scanned sequentially, or accessed directly if their RID is known,
- inserts place a new record in any page with enough free space,
- as the file grows or shrinks, pages are allocated and deallocated, so the DBMS must track free space.

The DBMS usually finds heap pages in one of two ways:

- **linked lists**: a header page points to a list of free pages and a list of data pages,
- **page directories**: directory pages store each data page location together with its remaining free space.



directory entry = page id + free-space summary

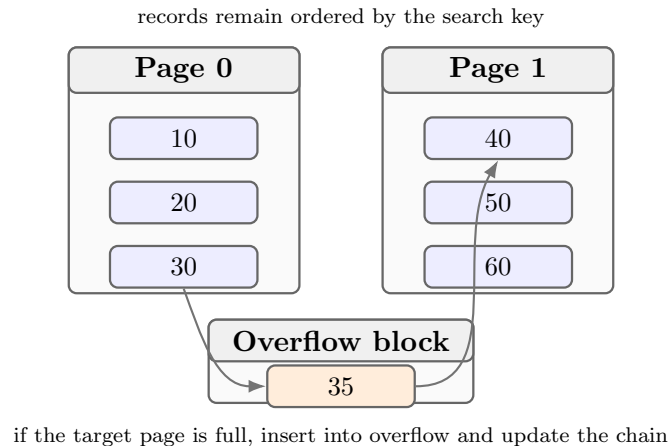
Heap files work well when most queries scan the file, because the DBMS does not maintain physical order.

1.4.2 Sorted (sequential) files

Search key. Attributes used to physically order records in a sorted file. They may be unique, but they do not have to be.

In a sorted file, records appear in search-key order.

- this is good for scanning the whole file in order,
- range queries are efficient because, after the first match, the DBMS can keep reading forward in order.



Keeping that order makes updates more expensive:

- **deletion:** systems often use pointer chains or tombstones so later records do not need to move right away,
- **insertion:** the DBMS finds the right position; if the target page has room, it inserts there, otherwise it uses an overflow page and updates the chain,
- over time, overflow pages accumulate, so the file must sometimes be reorganized to restore physical order.

1.4.3 Heap files versus sorted files

Which organization is better depends on the workload. Heap files make updates cheap. Sorted files make ordered search cheap. To compare them, we use a simple cost model.

Simplified I/O cost model. Measure cost only in page I/Os. In this model, I/O dominates CPU work, so we ignore buffering, prefetching, and other system effects. We also focus only on the average case.

Assumptions:

- B is the number of data pages in the file.
- Each operation inserts or deletes a single record.
- An equality search returns exactly one matching record.
- In a heap file, insertion appends at the end of the file.
- In a sorted file, searches use the file-ordering attribute and deletions keep the file compacted.

The main trade-off is:

- scans cost about the same in both organizations,
- sorted files are much better for equality and range search on the ordering attribute,
- heap files are much better for inserts and usually for deletes, because they avoid shifting many records.

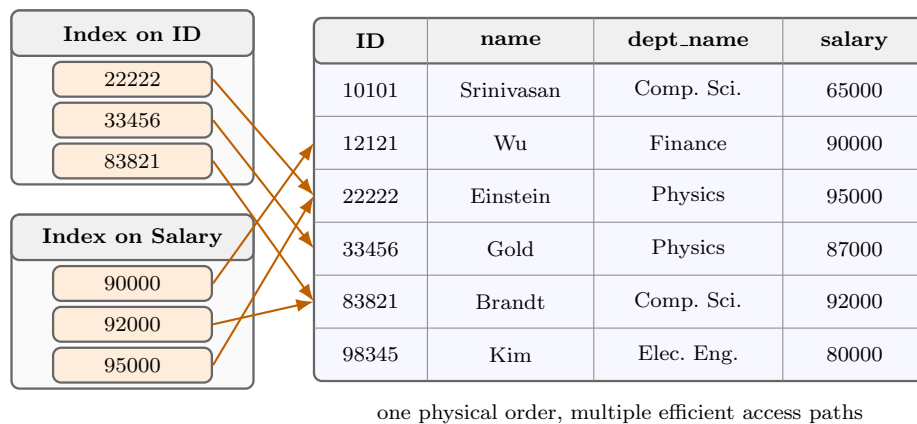
A sorted file gives one very good access path, but it pays heavily for updates. This is why indexes are useful.

Operation	Heap file	Sorted file	Notes
Scan all records	B	B	$B = \#$ of data pages
Equality search	$0.5B$	$\log_2 B$	assumes exactly one matching record
Range search	B	$\log_2 B + \# \text{match pages}$	after the first match, keep reading forward
Insert	2	$\log_2 B + 2 \cdot (B/2)$	must read and rewrite about half the file
Delete	$0.5B + 1$	$\log_2 B + 2 \cdot (B/2)$	search first, then pay the same shifting cost

1.5 Indexing

A sorted file helps only for one physical order. If the file is sorted on **ID**, then lookups on **ID** are efficient, but predicates on **salary** or **dept_name** may still need a scan.

Index. An auxiliary access structure that maps search-key values to records or pages, so the DBMS can answer selections with fewer I/Os than a full file scan.



Indexes matter because:

- they give the same relation several efficient access paths,
- they speed up selections on the index's search-key fields,
- the search key does not need to be unique,
- any subset of relation attributes can be used as a search key,
- inserts, deletes, and updates must also maintain the affected indexes.

1.5.1 Index search conditions

Index search condition. A predicate of the form $\langle \text{search key}, \text{comparison operator} \rangle$.

Examples:

- $\text{dept_name} = \text{"CS"}$: search key = **dept_name**, operator = equality.
- $\text{gpa} > 3$: search key = **gpa**, operator = greater-than.

Equality predicates ask for one exact key value. Inequality predicates such as $<$, $\leq$, $>$, and $\geq$ define ranges, so they benefit most from ordered access paths.

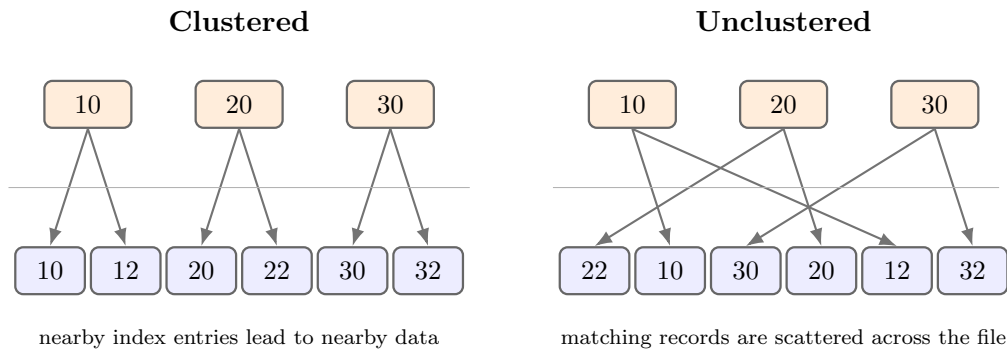
1.6 Index classification

Indexes are commonly described along four independent axes:

- clustered vs. unclustered,
- dense vs. sparse,
- key vs. non-key search key,
- data file ordered vs. not ordered on the indexing field.

1.6.1 Clustered and unclustered indexes

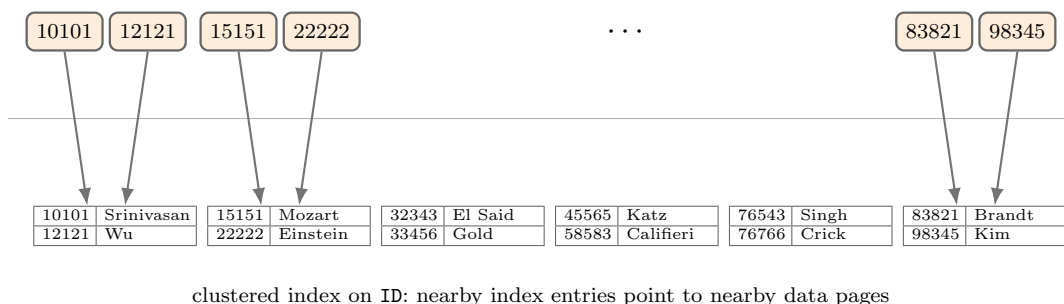
Clustered vs. unclustered. If the physical order of the data records follows the order of the index entries, the index is **clustered**. Otherwise it is **unclustered**.



This difference matters because:

- a clustered index keeps matching records on nearby pages, so range queries are efficient,
- an unclustered index may jump to many unrelated pages, so large result sets become expensive,
- a file can have many unclustered indexes, but only one clustered physical order at a time.

Some textbooks use *primary* for clustered and *secondary* for unclustered indexes. That vocabulary is not universal across DBMSs, so the physical term *clustered* is safer.



In the example above, the data file is ordered on ID. Nearby index entries lead to nearby data pages, which is exactly the clustered case.

1.6.2 Dense and sparse indexes

Dense index. Has an index entry for every search-key value that appears in the data file.

Sparse index. Has entries for only some search-key values, usually one entry per data page or page range. A sparse index uses less space and needs fewer updates, but it works well only when the data file is ordered on the search key. After the sparse index points to the right page, the DBMS still searches within that page.

Range scans: why clustering matters

For range predicates, the difference between clustered and unclustered indexes becomes much larger:

- with a **clustered** index, cost is roughly the number of **data pages** containing matching records,
- with an **unclustered** index, cost is closer to the number of **matching index entries**, because each match may lead to a different data page.

So the trade-off is:

- **clustered advantage**: very efficient for range queries and ordered scans,
- **clustered drawback**: expensive to maintain, because updates disturb the physical order,
- **unclustered advantage**: easy to add as an extra access path without reorganizing the data file,
- **unclustered drawback**: large result sets can cause many random I/Os.

Dense index on a key attribute

If the search key is a key, each value identifies one record. Then a dense index has one entry per record, and each entry can point directly to one RID.

Dense index	ID	name	dept_name	salary
10101	10101	Srinivasan	Comp. Sci.	65000
12121	12121	Wu	Finance	90000
15151	15151	Mozart	Music	40000
22222	22222	Einstein	Physics	95000
32343	32343	El Said	History	60000
33456	33456	Gold	Physics	87000
45565	45565	Katz	Comp. Sci.	75000
58583	58583	Califieri	History	62000
76543	76543	Singh	Finance	80000
76766	76766	Crick	Biology	72000
83821	83821	Brandt	Comp. Sci.	92000
98345	98345	Kim	Elec. Eng.	80000

dense index on a key: one index entry per record

Dense index on a non-key attribute

If the file is sorted on a non-key attribute, equal values appear together. A dense index can then keep one entry for each **distinct** search-key value and point to the first record of that group.

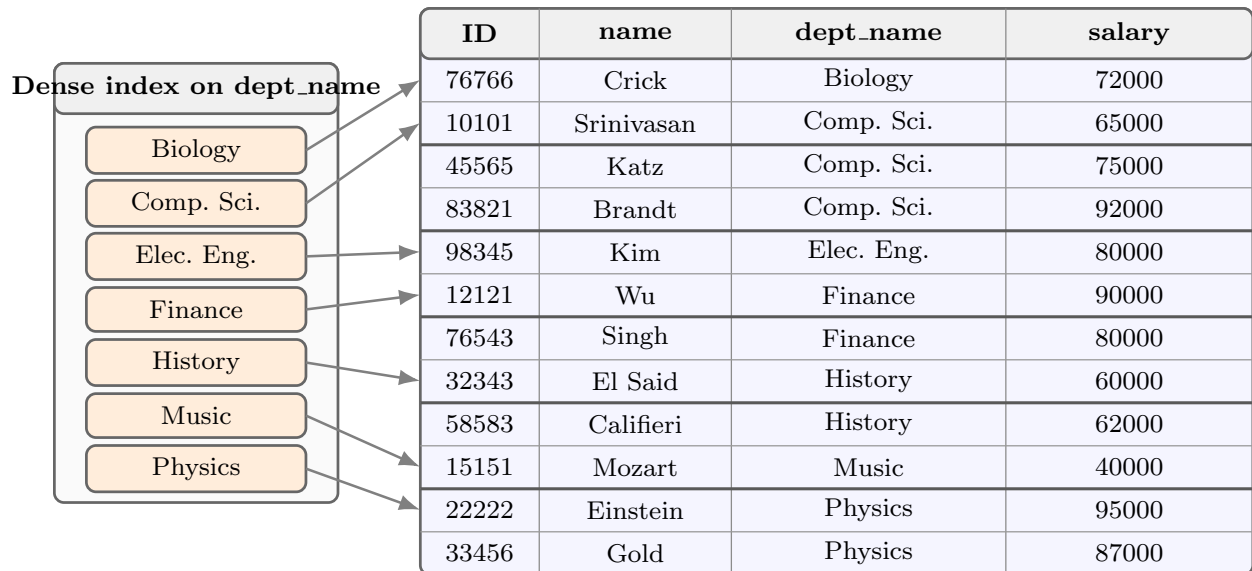
Sparse index files

Sparse indexes are most useful when the data file is already ordered on the search key. A common design keeps one anchor entry per data page.

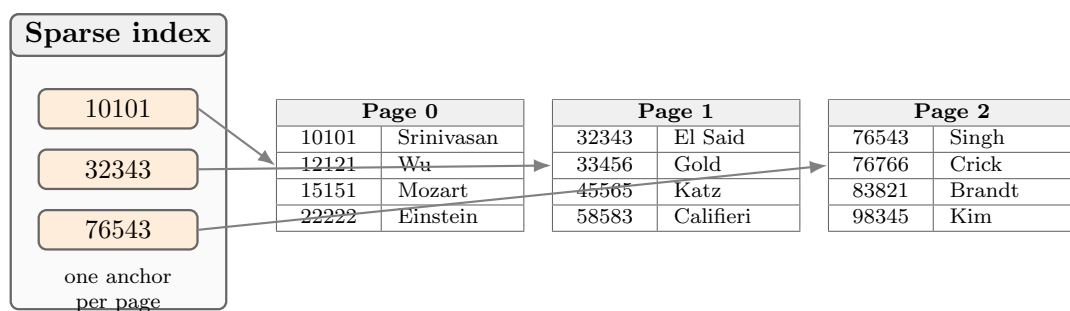
To locate a record with search-key value K :

- find the index entry with the largest key value $\leq K$,
- follow that pointer to the corresponding data page,
- scan forward in the file until K is found or passed.

Because this method relies on sequential order in the data file, a sparse index is normally clustered.



one entry per distinct search-key value, pointing to the first matching record



to search for K , jump to the page with the largest anchor key $\leq K$ and scan forward

1.6.3 Key and non-key search keys

If the search key is a key, each value identifies at most one record, so an index entry can point to one RID directly. If the search key is not a key, duplicates are possible, so one entry may need to point to a list or bucket of RIDs.

Physical order also matters. If the file is ordered on the indexing field, clustered and sparse organizations become possible. If it is not, the index is typically dense and unclustered.

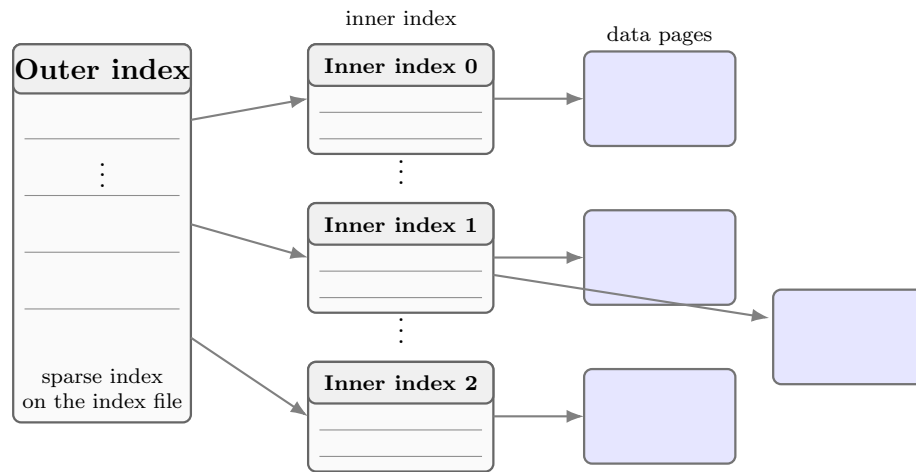
1.6.4 Multilevel indexes

If an index itself spans many pages, finding the right index page can become expensive.

Multilevel index. Treat the index file as a sorted file and build a sparse index on top of it. Repeat this until the top level is small enough to search very cheaply.

In a two-level design:

- the **inner index** is the ordinary index on the data file,
- the **outer index** is a sparse index on the pages of the inner index.



if one index is too large, build a sparse index on top of it

Search descends level by level until it reaches the target data page. If the outer index also becomes too large, another level can be added. The downside is that insertions and deletions may force updates at several levels.

1.6.5 Secondary indexes

Secondary index. An index whose search key does not define the physical order of the data file. In practice, this means the index is unclustered.

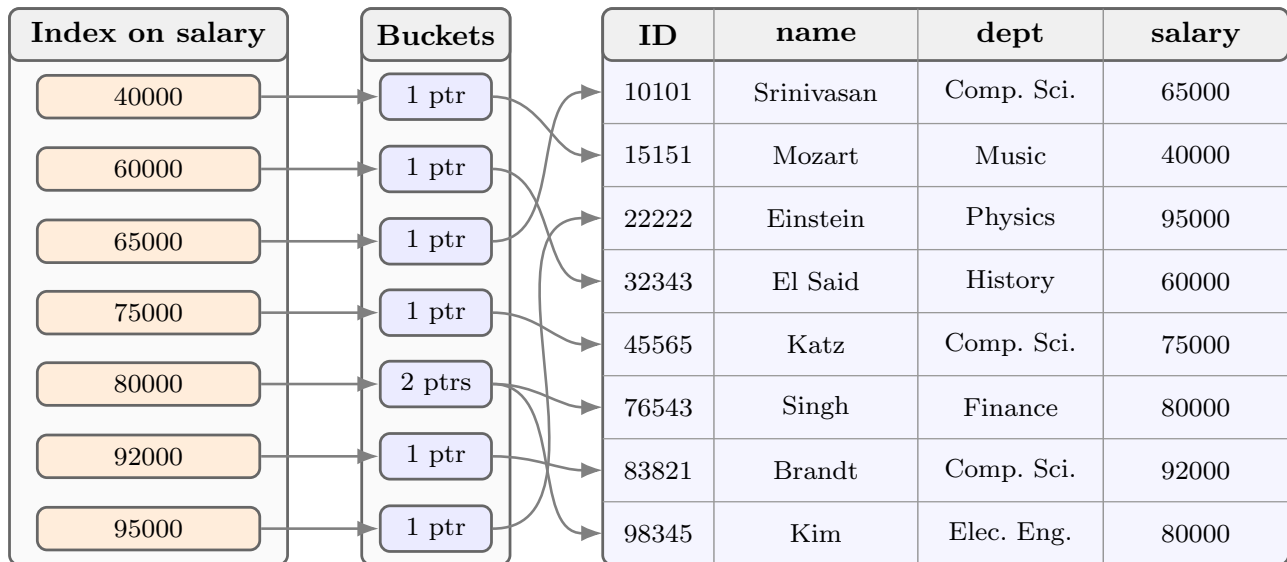
For a non-key search key, a secondary index usually works as follows:

- one index entry is stored for each search-key value,
- the index entry points to a bucket (or list) of RIDs,
- the bucket contains pointers to all matching records.

Secondary indexes on unsorted columns must be dense. Otherwise some matching records would not be reachable from the index.

1.7 B+-tree index files

Sequential dense and sparse indexes explain the core ideas clearly, but real DBMSs usually prefer B^+ -trees because they stay balanced under updates.

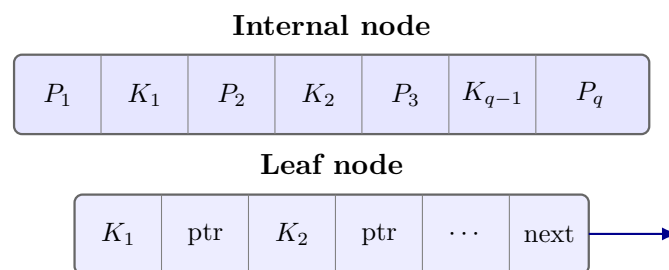


secondary index on an unsorted field: one entry per value, pointing to a bucket of record pointers

B+-tree. A balanced search tree whose nodes are page-sized blocks. Internal nodes store separator keys and child pointers; leaf nodes store sorted search-key entries and are linked left-to-right for efficient range scans.

For a tree of order n :

- all paths from the root to a leaf have the same length,
- each non-root internal node has between $\lceil n/2 \rceil$ and n children,
- each leaf has between $\lceil (n-1)/2 \rceil$ and $n-1$ key values,
- if the root is not a leaf, it has at least 2 children,
- if the root is also a leaf, it may contain between 0 and $n-1$ values.

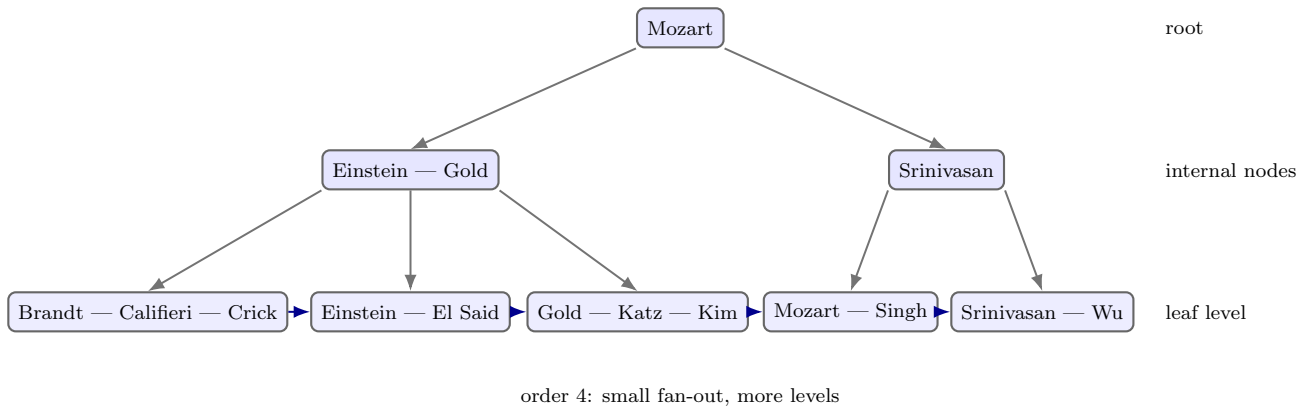
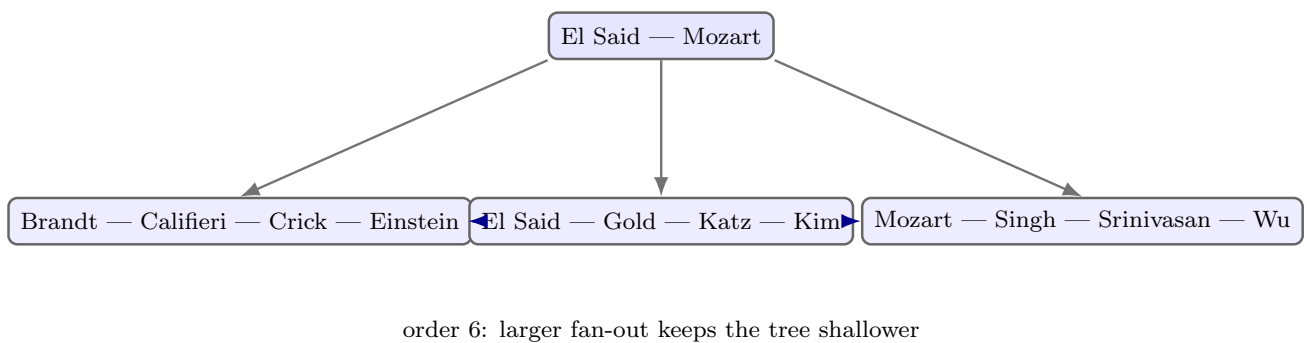


internal nodes route the search; leaves store actual entries and are linked for range scans

If an internal node has child pointers $P_1, \dots, P_q$, it stores separator keys $K_1, \dots, K_{q-1}$. Each key K_i separates the subtree reached through P_i from the subtree reached through P_{i+1} .

Example: order 4

With a small order, each node has limited fan-out, so the tree becomes wider and deeper more quickly. The leaf level is linked in sorted order, so once the first matching leaf is found, a range scan can continue sequentially.

**Example: order 6**

A larger order means more keys and child pointers fit in each node, so the same data can be indexed with fewer levels. This is why disk-based trees aim for high fan-out.

1.7.1 Revisiting clustered and unclustered terminology

To keep the vocabulary straight:

- **clustered / primary index:** the file is physically ordered on the indexing field; index entries may point to blocks or to individual records; the index can be sparse or dense,
- **unclustered / secondary index:** the file is not physically ordered on the indexing field; index entries point to records or to buckets of record pointers; the index must be dense.

So the core question is always physical order: does the data file itself follow the search-key order, or not?

1.8 Hash-based versus tree-based indexes

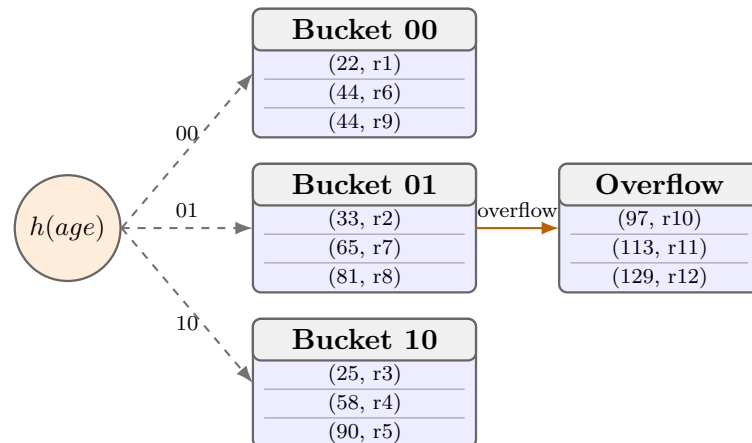
The two main index families solve different problems. Hash indexes optimize exact lookup. Tree indexes optimize ordered access.

1.8.1 Hash-based indexes

Hash-based index. Uses a hash function h on the search-key value to choose a bucket. Each bucket has one primary page and may have overflow pages if collisions accumulate.

Hash indexes are best for equality predicates:

- to evaluate **age** = 44, the DBMS computes $h(44)$ and reads the corresponding bucket,
- duplicate values naturally land in the same bucket,
- inserts and deletes usually affect one bucket only,
- the entries are **not** kept in sorted order.



hash index: equality lookup computes one bucket; collisions create overflow pages

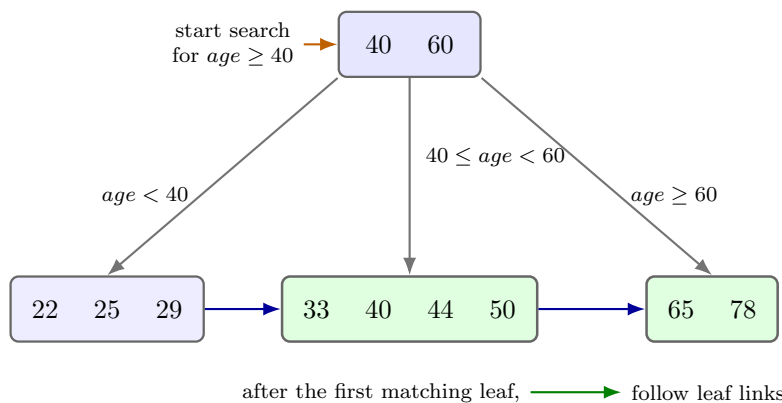
Because order is lost, hash indexes are poor for range predicates such as **age** > 40 or $3.0 \leq \text{gpa} \leq 3.5$. The DBMS would need to inspect many buckets, which is close to scanning the whole file.

1.8.2 Tree-based indexes

Tree-based index. Keeps entries sorted by search-key value. In practice, this is usually implemented as a B^+ -tree.

Tree indexes are good for range predicates:

- leaf entries are sorted by the search key,
- all root-to-leaf paths have the same length, so access remains balanced,
- after the first matching leaf is found, the DBMS follows the leaf links to continue the scan.



A tree index is more flexible: the same structure supports equality search, ordered scans, prefix lookups on composite keys, and range scans.

1.8.3 Indexing decisions follow queries

```
SELECT E.dno
FROM Employees E
WHERE E.age > 40
```

This query already suggests the right index family:

- `age > 40` is a range predicate, so a tree index on `age` is the natural choice,
- a hash index on `age` would not help, because it only groups equal values,
- if many qualifying tuples are expected, a **clustered** tree index is much better than an unclustered one,
- if the predicate matches a large fraction of the relation, a sequential scan may still be cheaper than using the index.

Selective predicate. Matches a small fraction of the relation. Indexes help most in this case; if most records qualify, scanning is often cheaper.

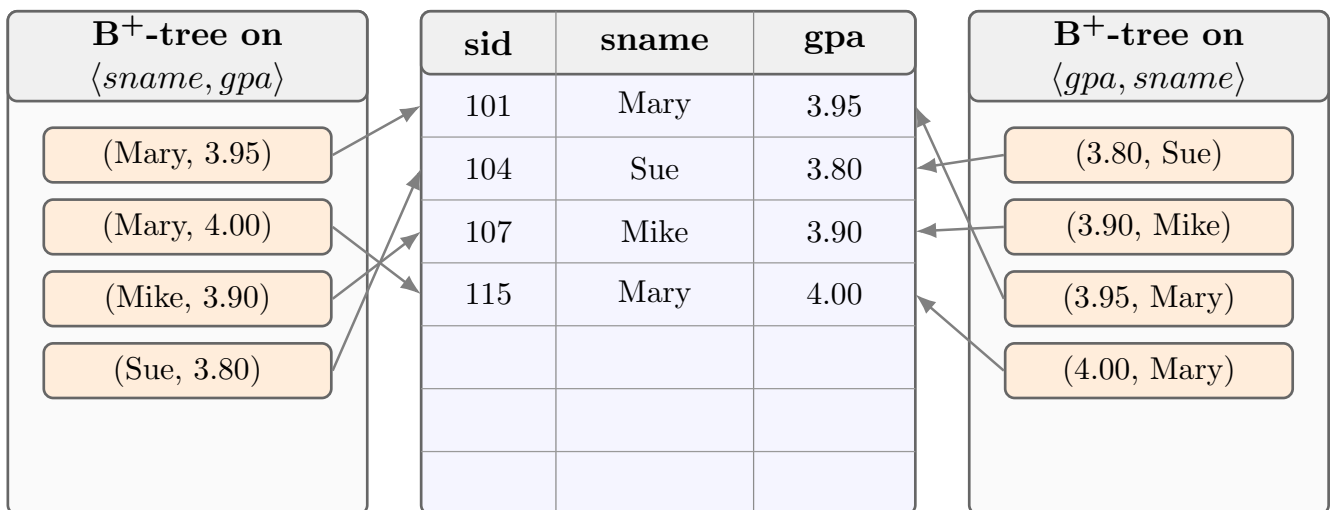
1.9 Choosing a search key

```
SELECT sid
FROM Student
WHERE sname = 'Mary'
AND gpa > 3.9
```

A good search key depends on which part of the predicate does the filtering:

- **index on sname:** useful because `sname = 'Mary'` is an equality condition. A hash index is a reasonable choice if this kind of lookup dominates.
- **index on gpa:** useful because `gpa > 3.9` is a range condition. This should be a tree index.
- **index on $\langle \text{sname}, \text{gpa} \rangle$:** often the best single index for this query. A tree index first narrows to `sname = 'Mary'`, then scans only the qualifying GPA values inside that group.

Lexicographic order. In a composite tree index on $\langle A, B \rangle$, entries are sorted first by A and, among equal A values, by B .



same attributes, different order: the useful predicates change with the lexicographic order

Field order matters. An index on $\langle \text{sname}, \text{gpa} \rangle$ is much better for the query above than an index on $\langle \text{gpa}, \text{sname} \rangle$.

1.9.1 Composite search keys

Composite search key. An index whose search key is a tuple of attributes rather than one single attribute.

For composite keys, the shape of the predicate matters:

- **full equality:** with `age = 12 AND sal = 75`, both a hash index and a tree index on $\langle \text{age}, \text{sal} \rangle$ can work.
- **prefix equality:** with `age = 12`, a tree index on $\langle \text{age}, \text{sal} \rangle$ still works because all entries with the same age are adjacent. A hash index on the full pair does not.
- **prefix equality + range:** with `age = 12 AND sal > 20`, a tree index on $\langle \text{age}, \text{sal} \rangle$ is ideal. An index on $\langle \text{sal}, \text{age} \rangle$ is much less useful for that query.

Leftmost-prefix rule. A composite tree index on $\langle A, B, C \rangle$ can be used efficiently only when the query constrains a prefix of the key: A , or A and B , or A , B , and C .

1.9.2 Composite keys and index-only evaluation

```
SELECT AVG(E.sal)
FROM Employees E
WHERE E.age = 25
      AND E.sal BETWEEN 3000 AND 5000
```

For this query:

- the right structure is a tree index, not a hash index, because **BETWEEN** is a range predicate,
- the better order is $\langle \text{age}, \text{sal} \rangle$, because the search starts with one exact age and then scans only the salary interval inside that age group,
- an index on $\langle \text{sal}, \text{age} \rangle$ would first scan the salary interval for *all* ages and then filter `age = 25`,
- the query needs only `sal`, which already appears in the composite key.

Index-only evaluation. If every attribute needed by the query is already present in the index entries, the DBMS can answer the query from the index alone, without fetching the full data records.

This is also called a **covering index**. It is especially attractive for aggregates such as **AVG**, **COUNT**, **MIN**, and **MAX**.

1.10 System catalogs

System catalog. The DBMS's metadata repository. It stores information about relations, attributes, indexes, views, constraints, statistics, and access control.

The catalog typically stores:

- **for each relation:** relation name, backing file, file organization, attribute names and types, and integrity constraints,
- **for each index:** index name, structure (B^+ -tree, hash, ...), search-key fields, and clustering status,
- **for each view:** view name and definition,
- **for the optimizer and system:** statistics such as relation cardinality, page count, average row length, plus authorization and other metadata.

Catalogs are themselves stored as relations, so the DBMS can query them like ordinary tables.

1.10.1 Example: Oracle system catalogs

Oracle exposes catalog information through views such as `USER_TABLES` and `USER_TAB_COLUMNS`.

```
SELECT *
FROM student
```

student

SID	NAME	AGE
A1234	John	23
B1	Xavier	24
A21341	Scott	21
C12948291	Benjamin	25
1948	Ben	29

```
SELECT table_name, num_rows, blocks, avg_row_len
FROM user_tables
```

TABLE_NAME	NUM_ROWS	BLOCKS	AVG_ROW_LEN
STUDENT	5	5	20

```
SELECT table_name, column_name, data_type, data_length
FROM user_tab_columns
WHERE table_name = 'STUDENT'
```

TABLE_NAME	COLUMN_NAME	DATA_TYPE	DATA_LENGTH
STUDENT	SID	CHAR	10
STUDENT	NAME	VARCHAR2	32
STUDENT	AGE	NUMBER	22

These catalog views are what the optimizer and storage manager rely on: they describe the schema, give rough size estimates, and tell the system which access paths already exist.